Instituto Politécnico Nacional

Escuela Superior de Cómputo

Unidad Académica:

Sistemas Distribuidos

Profesor:

Carreto Arellano Chadwick

Práctica 05: "Integración de Broker RMI para observabilidad en arquitectura de múltiples clientes y múltiples servidores"

Alumna:

Maldonado Bedolla Daniela del Carmen

Grupo:

7CM1

23 de marzo de 2026

Índice

INTRODUCCIÓN **3**

DESARROLLO **4**

Problemática 4
Propuesta de solución 4
Integración del broker en la arquitectura distribuida 6

IMPLEMENTACIÓN **8**

Generalidades *8*
Nota *9*
Servidor *9*
Cliente *11*
Coordinador *12*
Broker *14*
Resumen del funcionamiento 15

EJECUCIÓN Y RESULTADOS **16**

Ejecución de la simulación 16

RESULTADOS **20**

Broker RMI (monitoreo global) 20
Coordinador global 20
Servidor S1 (SALA-01) 21
Cliente del servidor S1 ..... *21*
Servidor S2 (SALA-02) 22
Cliente del servidor S2 ..... *22*

COMPARACIÓN DE RESULTADOS CON PRÁCTICAS ANTERIORES **24**

CONCLUSIONES **26**

ANEXO **27**

BrokerService.java 27
Código del coordinador 28
Código del servidor 29
Código de cliente 30

INTRODUCCIÓN

La evolución de un sistema concurrente local hacia una arquitectura distribuida escalable representa una transición natural en el diseño de software moderno. En las primeras prácticas, la venta de boletos se resolvió dentro de un solo proceso con múltiples hilos, lo que permitió analizar exclusión mutua, condiciones de carrera y consistencia sobre memoria compartida. Después, al migrar al esquema cliente/servidor, el control del recurso crítico se concentró en el servidor y la sincronización pasó a realizarse mediante intercambio de mensajes por red.

En esta práctica, el modelo se amplía a un entorno de múltiples clientes y múltiples servidores, donde cada servidor administra una venta independiente y un coordinador global define el inicio simultáneo de la fase de compra. Este enfoque introduce una condición de arranque distribuida: la venta no depende únicamente del estado local de un nodo, sino del estado agregado del sistema.

Sobre esta arquitectura se integra un broker para observabilidad. Un broker es un componente intermediario especializado en recibir, concentrar y exponer eventos del sistema sin alterar la lógica principal de negocio. En este caso, el broker no decide compras ni asigna asientos; su función es registrar eventos como conexión de servidores, conexión de clientes, progreso de venta y finalización de cada nodo, para ofrecer una vista consolidada en tiempo real.

La implementación del broker se realizó con Java RMI (Remote Method Invocation). RMI es una tecnología de Java que permite invocar métodos de objetos remotos como si fueran llamadas locales, encapsulando la comunicación de red, serialización y despacho remoto. Con ello, los procesos externos pueden notificar eventos al broker mediante una interfaz remota clara y tipada, simplificando la integración de un servicio distribuido de monitoreo.

El uso de Java aporta un entorno maduro para objetos distribuidos, bibliotecas estándar para RMI y construcción rápida de interfaces gráficas para supervisión operativa. Así, Java cumple un doble papel en esta práctica: capa de comunicación remota del broker y capa de visualización del estado global.

En conjunto, esta etapa consolida una arquitectura distribuida con sincronización global, control concurrente de recursos finitos y observabilidad centralizada, sentando bases para extensiones futuras como desacoplamiento por servicios, trazabilidad avanzada y estrategias de tolerancia a fallos.

Desarrollo

Problemática

En la versión original del modelo cliente/servidor, la dinámica era una sola venta centralizada: una “flecha” de tráfico desde múltiples plataformas cliente hacia un único servidor que administraba el inventario. En ese esquema, muchos clientes actuaban como puntos de acceso con alta afluencia de compradores concurrentes, y todos competían por un conjunto finito de asientos controlado por una sola autoridad de estado.

Al escalar el escenario, el problema se extiende a la venta de N fechas en paralelo. Esto implica N servidores de venta (uno por fecha) y múltiples clientes que generan afluencia para cada servidor según la fecha objetivo. En este contexto, los retos ya no son solo de concurrencia local por servidor, sino también de coordinación global entre servidores para iniciar y operar ventas simultáneas sin inconsistencias.

Los riesgos principales pasan a ser:

- Competencia intensa por asientos dentro de cada servidor (doble intento de reserva/compra sobre el mismo recurso).
- Desfase entre estado percibido por clientes y estado real del servidor por latencia o reintentos.
- Reservas no confirmadas que pueden “atorar” asientos si no se liberan automáticamente.
- Alta tasa de intentos fallidos en la etapa final de cada venta (pocos asientos restantes).
- Desalineación de inicio entre servidores si no existe sincronización global para arrancar todas las fechas al mismo tiempo.
- Persistencia de conexiones abiertas al finalizar, afectando cierre limpio de procesos distribuidos.

En síntesis, la problemática evoluciona de “muchos hilos contra un inventario” a “muchos clientes y muchos servidores coordinados”, donde se debe garantizar consistencia por venta y sincronización del sistema completo.

Propuesta de solución

La solución se estructuró en una arquitectura multi-cliente / multi-servidor con coordinación central:

- Cada servidor administra una venta independiente (una fecha) y mantiene autoridad total sobre sus asientos.
- Cada cliente (punto de acceso) se dirige explícitamente a un servidor/fecha (sale_id) y genera afluencia mediante hilos compradores.
- Un coordinador central registra servidores y clientes conectados por fecha, y sincroniza el arranque global con señal GLOBAL_START.
- El servidor conserva control estricto de estados de asiento (FREE, RESERVED, SOLD) y exclusión mutua para evitar doble venta.
- Las reservas son temporales (TTL); si no se confirman, se liberan automáticamente y regresan a disponibilidad.
- La compra solo la puede confirmar el mismo comprador que creó la reserva (validación de propiedad).
- Cada venta cierra por condición de término (agotamiento, fin de clientes o inviabilidad de demanda elegible), manteniendo consistencia de conteos.
- Se incorporó visualización en tiempo real:
    - en servidor: progreso de la venta por asientos;
    - en coordinador: estado de servidores y afluencia de clientes por fecha.
- Se reforzó el cierre lógico de conexiones:
    - notificación explícita de desconexión servidor-coordinador;
    - cierre ordenado de sockets/streams en cliente, servidor y coordinador;
    - configuración de hilos daemon para evitar bloqueos en apagado.

Con este enfoque se obtiene escalabilidad horizontal por fecha, sincronización global de arranque, integridad transaccional por servidor y finalización limpia de ejecución en todo el sistema distribuido.

usa un modelo cliente/servidor donde el servidor es la autoridad total del sistema:

- El cliente crea los 2,000 hilos compradores y cada hilo manda solicitudes por socket.
- El servidor valida todo: solicitud de asiento, apartado y compra final.
- Los asientos se manejan con estados (FREE, RESERVED, SOLD) y control de exclusión para evitar doble venta.
- El apartado tiene tiempo límite (TTL); si no se confirma a tiempo, el asiento se libera automáticamente.

- La compra solo se permite al mismo comprador que hizo la reserva (validación de dueño de reserva).
- Al llegar a 1,000 vendidos, el servidor marca venta terminada y deja de aceptar nuevas compras.

Además, se agregó visualización en el servidor y métricas de tiempo para observar el comportamiento del sistema.
Con este enfoque se logra que no haya ventas duplicadas, que el conteo final sea correcto y que el cierre de ejecución sea limpio tanto en servidor como en cliente.

Integración del broker en la arquitectura distribuida

La incorporación de un broker en esta problemática responde a una necesidad de desacoplar la operación de venta del monitoreo global del sistema. En un escenario con múltiples servidores de venta y múltiples clientes concurrentes, el broker funciona como un punto de concentración de eventos: recibe estados relevantes de cada nodo, normaliza la información y la presenta de forma unificada para facilitar supervisión, diagnóstico y trazabilidad.

Desde un enfoque de arquitectura, existen dos formas principales de usarlo:

1. Broker como intermediario de observabilidad (enfoque implementado): El broker no participa en la decisión de negocio (reserva/compra), sino que consume eventos operativos para mostrar estado del sistema en tiempo real.
2. Broker como intermediario de tráfico (enfoque potencial): El broker podría colocarse entre clientes y servidores para enrutar solicitudes, balancear carga, aplicar políticas de backpressure, priorizar colas, o aislar fallos entre nodos.

En términos de analogía real, esto se parece a plataformas de venta masiva donde existe una capa central que consolida estado de múltiples centros de venta y canales de acceso, permitiendo observar demanda, disponibilidad y comportamiento operativo sin que cada componente de negocio tenga que conocer internamente a todos los demás.

Respecto a escalabilidad, el patrón permite crecer horizontalmente por ventas/fechas, y también evolucionar hacia esquemas con múltiples brokers o capas de agregación regional. De esta forma, la solución puede pasar de un monitoreo local a una operación distribuida de mayor alcance sin rediseñar por completo la lógica transaccional de los servidores de venta.

Alcance específico de esta implementación:

En esta práctica, el broker se integró en modo observabilidad, con los siguientes objetivos concretos:

- Mantener al servidor de venta como autoridad total del recurso (asientos y transacciones).
- Conservar al coordinador como responsable de la sincronización de arranque global (GLOBAL_START).
- Recibir en el broker eventos de:
    - registro de servidor,
    - conexión de clientes,
    - progreso de asientos vendidos (valor absoluto y porcentaje),
    - finalización de venta por servidor.
- Mostrar en GUI central el estado por servidor con semántica operativa:
    - Conectado,
    - Esperando clientes,
    - Venta en curso,
    - Finalizado,
      junto con avance de venta en la misma fila.
- Permitir evolución futura hacia un broker de enrutamiento y balanceo, sin alterar la base transaccional ya implementada.

Implementación

Generalidades

La implementación actual extiende el problema original hacia un escenario de N ventas simultáneas y añade explícitamente una capa de observabilidad distribuida mediante broker. En lugar de una sola venta centralizada, el sistema permite ejecutar múltiples servidores de boletos en paralelo, donde cada servidor representa una venta independiente (por ejemplo, distintas fechas). Cada venta conserva su propio inventario, su propia concurrencia interna y su propio conjunto de clientes, mientras el coordinador mantiene la sincronización global de arranque.

El elemento distintivo de esta práctica es la integración del broker RMI como componente transversal de monitoreo. El broker recibe eventos de registro de servidores, conexión de clientes, avance de venta y finalización por cada sale_id, y consolida esa información en una vista única en tiempo real. Con ello, el sistema deja de observarse solo desde cada nodo aislado y pasa a tener una lectura global del comportamiento distribuido. Este enfoque desacopla la lógica de negocio de la lógica de supervisión y prepara la arquitectura para escalar en diagnóstico, trazabilidad y operación.

A nivel visual, la solución incorpora tres perspectivas complementarias:

- Interfaz local por servidor para visualizar su simulación de asientos y dinámica de venta.
- Interfaz del coordinador con grid dinámico según el número de servidores, mostrando estado global y afluencia de clientes por venta.
- Interfaz del broker con estado operativo por servidor (conectado, esperando clientes, venta en curso, finalizado) y progreso de asientos vendidos.

En la versión implementada, el foro se modela como una matriz de 30 filas x 50 columnas, para un total de 1,500 asientos por servidor. El estado de cada asiento se maneja mediante tres valores: FREE, RESERVED y SOLD, lo cual permite reservas temporales, confirmación posterior de compra y liberación automática por expiración (TTL).

También se incorporan zonas y tipos de comprador para simular restricciones de acceso reales:

- Zonas: Platino, Preferente, Normal.
- Tipos: platino, preferente, normal.

Cada tipo de comprador tiene reglas de elegibilidad sobre zonas permitidas, por lo que el sistema no solo resuelve concurrencia sobre recursos finitos, sino también validación de acceso, gestión de demanda por perfil y cierre por inviabilidad cuando los asientos restantes ya no son compatibles con los tipos de clientes conectados.

Nota

Debido a la extensión de los programas, la implementación se explica a través dxe bloques de código que ejemplifican las modificaciones realizadas para el funcionamiento de la práctica.

Servidor

El servidor mantiene la autoridad sobre inventario y transacciones, pero ahora publica su estado operativo al broker. Se añadieron notificaciones de registro, progreso de venta y finalización, sin delegar la lógica crítica de compra.

Bloque 1: envío genérico de eventos al broker

```python
def notify_broker_event(broker_config, event_type, sale_id, **extra_args):
    command = ["java", "-cp", broker_config["java_cp"], "broker.BrokerEventCli",
               "--host", broker_config["host"], "--port", str(broker_config["port"]),
               "--bind", broker_config["bind"], "--event", event_type, "--sale-id", sale_id]

    if event_type == "SERVER_REGISTERED":
        command.extend(["--server-host", str(extra_args.get("server_host", "127.0.0.1")),
                        "--server-port", str(int(extra_args.get("server_port", 0))),
                        "--expected-clients", str(int(extra_args.get("expected_clients", 0)))])
    elif event_type == "SALES_PROGRESS":
        command.extend(["--sold", str(int(extra_args.get("sold", 0))),
                        "--total", str(int(extra_args.get("total", 0)))])
    elif event_type == "SERVER_FINISHED":
        command.extend(["--summary", str(extra_args.get("summary", ""))])

    subprocess.run(command, check=True, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL, timeout=6)
```

Este bloque encapsula toda la integración con el broker en una sola función. Según el tipo de evento, agrega los parámetros necesarios y ejecuta el cliente Java

(BrokerEventCli) por subprocess. Así, el servidor Python mantiene su diseño original y solo “publica” eventos.

Bloque 2: monitoreo periódico de progreso

```python
def monitor_progress_to_broker(ticket_state, broker_config, sale_id):
    last_sold = -1
    last_sent_at = 0.0
    while not ticket_state.sold_out_event.is_set():
        now = time.monotonic()
        with ticket_state.meta_lock:
            sold = int(ticket_state.sold_count)

        changed = sold != last_sold
        heartbeat_due = (now - last_sent_at) >= 2.0
        if changed or heartbeat_due:
            notify_broker_event(broker_config, "SALES_PROGRESS", sale_id, sold=sold, total=TOTAL_ASIENTOS)
            last_sold = sold
            last_sent_at = now
        time.sleep(0.20)
```

Este hilo reporta avance de asientos vendidos sin saturar la red: envía cuando cambia el valor de sold_count o por heartbeat cada 2 segundos. El broker recibe tanto actualización reactiva como señal de vida del servidor.

Bloque 3: cierre de venta y notificación final

```python
def monitor_sold_out(ticket_state, coordinator_client=None, broker_config=None, sale_id=None):
    ticket_state.sold_out_event.wait()
    # ... cálculo de finish_summary ...
    summary_text = f"sold={finish_summary['sold_count']}/{finish_summary['total_seats']}, reason={finish_summary['clo
    notify_broker_event(broker_config, "SERVER_FINISHED", sale_id, summary=summary_text)
```

Cuando la venta termina por cualquier condición, este bloque construye un resumen consolidado y envía el evento final al broker. Con eso, la GUI del broker puede cerrar el ciclo visual del servidor y marcarlo como finalizado.

Bloque 4: configuración explícita de broker

```python
broker_config = {
    "host": args.broker_host,
    "port": args.broker_port,
    "bind": args.broker_bind,
    "java_cp": args.broker_java_cp
}
```

Este bloque centraliza los parámetros de conexión al broker para reutilizarlos en todas las notificaciones. Facilita pruebas, cambios de entorno y evita duplicar valores en múltiples funciones.

Cliente

El cliente sigue generando carga concurrente de compradores, pero ahora también informa al broker cuándo un punto de acceso se conecta y cuántos compradores representa.

Bloque 1: notificación de conexión al broker

```
def notify_broker_client_connected(broker_host, broker_port, broker_bind, broker_java_cp,
                                   sale_id, client_id, buyers_count):
    command = ["java", "-cp", broker_java_cp, "broker.BrokerEventCli",
               "--host", broker_host, "--port", str(broker_port), "--bind", broker_bind,
               "--event", "CLIENT_CONNECTED", "--sale-id", sale_id,
               "--client-id", client_id, "--buyers", str(int(buyers_count))]
    subprocess.run(command, check=True, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL, timeout=5)
```

Este bloque publica en el broker la incorporación de un cliente lógico al sistema distribuido. El dato de buyers_count permite al broker reflejar no solo conexión, sino también magnitud de afluencia por servidor.

Bloque 2: parámetros CLI del broker

```
parser.add_argument("--broker-host", default="127.0.0.1")
parser.add_argument("--broker-port", type=int, default=1099)
parser.add_argument("--broker-bind", default="BrokerService")
parser.add_argument("--broker-java-cp", default="out")
```

Estos argumentos permiten desacoplar la configuración de red del código fuente. El cliente puede ejecutarse en distintos escenarios sin modificar funciones internas, solo cambiando banderas de terminal.

Bloque 3: orden de notificación e inicio

```
if args.coordinator_host:
    notify_coordinator_client_connected(args.coordinator_host, args.coordinator_port, sale_id, client_id, args.buyers

notify_broker_client_connected(
    args.broker_host, args.broker_port, args.broker_bind, args.broker_java_cp,
    sale_id, client_id, args.buyers
)

register_and_wait_start(args.host, args.port, client_id, normalized_type, args.buyers)
```

Este bloque muestra la secuencia operativa: primero sincroniza con coordinador, luego reporta al broker y finalmente espera la señal global de inicio. Así, la trazabilidad en broker coincide con el flujo real de arranque.

Coordinador

El coordinador conserva su rol de barrera global, pero se reforzó la observabilidad textual en consola para identificar rápidamente cuántos servidores/clientes faltan antes de iniciar.

Bloque 1: log de registro de servidores

```
registered = result["registered_servers"]
expected_servers = result["expected_servers"]
remaining = expected_servers - registered
print(f"[Servidor] {sale_id} conectado. Servidores registrados: {registered}/{expected_servers}. Faltan {remaining}."
```

Este bloque aporta diagnóstico inmediato del progreso de infraestructura: muestra cuántos nodos de venta ya están listos y cuántos faltan para cumplir la condición global.

Bloque 2: log de registro de clientes

```
if expected is not None:
    remaining_clients = expected - connected
    print(f"[Cliente] {client_id} conectado a {sale_id}. Clientes: {connected}/{expected}. Faltan {remaining_clients}
else:
    print(f"[Cliente] {client_id} conectado a {sale_id} (servidor aún no registrado).")
```

Este bloque clarifica el avance por venta (sale_id) y permite detectar casos de orden no ideal (cliente reporta antes de servidor). Es útil para depuración de sincronización en ejecuciones distribuidas.

Bloque 3: countdown previo al arranque global

```
def broadcast_global_start_delayed(self, countdown_seconds=8):
    def _countdown_and_broadcast():
        print(f"[Coordinador] Iniciando cuenta regresiva de {countdown_seconds} segundos antes de GLOBAL_START...")
        for i in range(countdown_seconds, 0, -1):
            print(f"[Countdown] {i}...")
            time.sleep(1)
        print("[Countdown] 0 - ¡INICIANDO VENTAS GLOBALES!")
        self.broadcast_global_start()
    threading.Thread(target=_countdown_and_broadcast, daemon=True).start()
```

Este bloque separa la transición de estado “todos listos” del arranque efectivo de ventas. El retardo controlado facilita validar sincronización visual y comportamiento simultáneo de servidores.

Bloque 4: validación de condición global

```python
def _maybe_mark_global_start_locked(self):
    if self.global_started:
        return False
    if not self._all_clients_connected_locked():
        return False
    self.global_started = True
    for sale_id in self.handlers_by_sale:
        info = self.server_info_by_sale.get(sale_id)
        if info is not None:
            info["status"] = STATUS_ORANGE
    return True
```

Este bloque es la compuerta central de inicio: evita doble disparo de arranque y solo habilita transición cuando todos los servidores y sus clientes esperados están listos.

Broker

Se añadió un broker en Java RMI para observabilidad centralizada. Su función es recibir eventos remotos, consolidar estado por sale_id y exponerlo en GUI con estado en español y progreso de asientos vendidos.

Bloque 1: Contrato remoto del broker

```java
public interface BrokerRemote extends Remote {
    void serverRegistered(String saleId, String serverHost, int serverPort, int expectedClients) throws RemoteExcepti
    void clientConnected(String saleId, String clientId, int buyers) throws RemoteException;
    void salesProgress(String saleId, int soldSeats, int totalSeats) throws RemoteException;
    void serverFinished(String saleId, String summary) throws RemoteException;
    BrokerSnapshot getSnapshot() throws RemoteException;
}
```

Este contrato define las operaciones remotas oficiales del broker. Es la frontera de comunicación entre nodos Python y servicio Java, y asegura una API clara para observación distribuida.

Bloque 2: procesamiento de progreso de ventas

```java
@Override
public void salesProgress(String saleId, int soldSeats, int totalSeats) {
    synchronized (lock) {
        SaleStatus state = bySale.computeIfAbsent(saleId, this::newSale);
        state.soldSeats = Math.max(0, soldSeats);
        state.totalSeats = Math.max(0, totalSeats);
        if (!STATUS_RED.equals(state.status) && state.connectedClients > 0 && state.soldSeats > 0) {
            state.status = STATUS_ORANGE;
        }
        state.updatedAt = now();
    }
}
```

Este bloque actualiza progreso numérico de venta y permite transicionar a “Venta en curso” cuando ya existe actividad real de compra. El estado final se preserva en rojo cuando ya terminó.

Bloque 3: CLI puente Python→RMI

```java
switch (normalized) {
    case "SERVER_REGISTERED":
        remote.serverRegistered(saleId, serverHost, serverPort, expectedClients);
        break;
    case "CLIENT_CONNECTED":
        remote.clientConnected(saleId, clientId, buyers);
        break;
    case "SALES_PROGRESS":
        remote.salesProgress(saleId, sold, total);
        break;
    case "SERVER_FINISHED":
        remote.serverFinished(saleId, summary);
        break;
}
```

Este bloque traduce eventos de línea de comandos a invocaciones remotas reales. Es el punto clave que permite integrar procesos Python con el servicio RMI sin reescribir el sistema existente en Java.

Bloque 4: render de estado y barra de progreso

Este bloque materializa la observabilidad en GUI: combina estado cualitativo y progreso cuantitativo por servidor en una sola fila, permitiendo seguimiento en tiempo real durante toda la simulación.

```
int sold = Math.max(0, sale.soldSeats);
int total = Math.max(0, sale.totalSeats);
int percent = total > 0 ? Math.min(100, (int)Math.round((sold * 100.0) / total)) : 0;

soldProgressText.setText(sold + "/" + total + " asientos (" + percent + "%)");
soldProgressBar.setValue(percent);
statusLabel.setText(statusToLabel(sale));   // Conectado / Esperando clientes / Venta en curso / Finalizado
statusPanel.setBackground(statusToColor(sale)); // verde / amarillo / naranja / rojo
```

Resumen del funcionamiento

El broker se integra como una capa de observabilidad distribuida sobre la arquitectura cliente/servidor coordinada. Su función principal es recibir eventos de estado desde los nodos operativos (servidores y clientes) y consolidarlos por venta (sale_id) en una vista única en tiempo real.

A nivel técnico, se implementa con Java RMI, lo que permite invocar métodos remotos para registrar conexión de servidores, conexión de clientes, avance de venta (asientos vendidos y porcentaje) y finalización de cada servidor.

El broker no sustituye la lógica transaccional del servidor ni la sincronización global del coordinador; en cambio, las complementa con monitoreo centralizado, trazabilidad del ciclo completo de venta y diagnóstico operativo inmediato. Gracias a este desacoplamiento, el sistema conserva consistencia en la compra de boletos y, al mismo tiempo, mejora su capacidad de supervisión y escalabilidad para escenarios con múltiples ventas concurrentes.

Ejecución y Resultados

La validación se realizó en un escenario distribuido con dos ventas simultáneas, cada una gestionada por un servidor independiente, un coordinador global para sincronización de arranque y un broker RMI para observabilidad centralizada. A diferencia de prácticas anteriores, en esta ejecución el broker se levanta desde el inicio como componente obligatorio, con el fin de registrar en tiempo real el estado de servidores, conexión de clientes, avance de asientos vendidos y finalización de cada venta.

La prueba se ejecutó conforme al flujo definido en el archivo de instrucciones, iniciando con compilación y arranque del broker, seguido del coordinador, los dos servidores y finalmente dos clientes de carga. En esta configuración, el cliente del servidor SALA-01 genera 2000 compradores y el cliente del servidor SALA-02 genera 1500 compradores, lo que permite observar un comportamiento de alta concurrencia y contrastar el avance de ambas ventas en paralelo.

Este escenario permitió observar el flujo completo: registro de nodos, conexión de clientes, cuenta regresiva de inicio, evolución de estados y cierre de ventas. Los comandos exactos utilizados para la prueba se encuentran en INTRUCCIONES_DE_PRUEBA_2_SERVIDORES.txt, anexada en la entrega de esta práctica.

Ejecución de la simulación

Al iniciar el coordinador, la interfaz muestra ambos servidores en estado inicial sin registro. En esta etapa el sistema se encuentra en espera de infraestructura, sin condiciones aún para disparar la venta global.

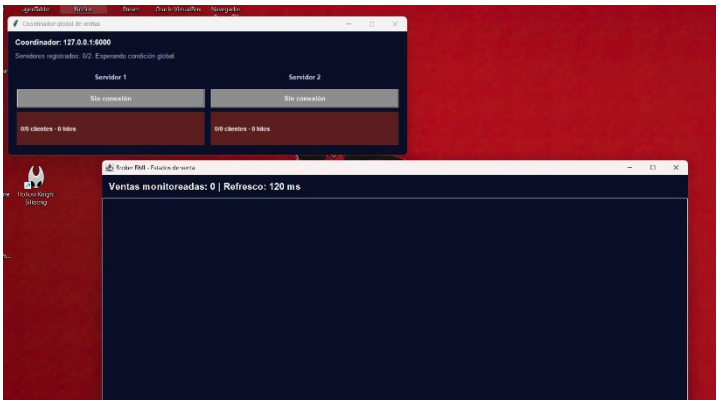

Figura 1. Estado inicial del coordinador antes del registro de servidores.

Con el arranque de los dos servidores, el coordinador actualiza su grid dinámico y refleja el registro progresivo de SALA-01 y SALA-02. En paralelo, el broker crea una fila por cada venta y muestra su transición de estado operativo.

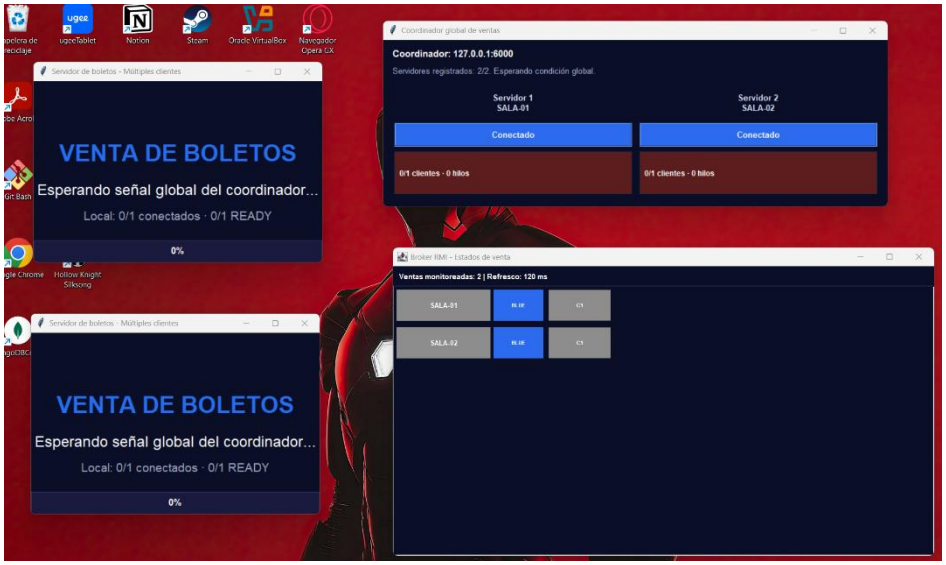

Figura 2. Registro de servidores y aparición de ventas en el broker.

Posteriormente, al conectarse los clientes de acceso, el coordinador reporta la afluencia por venta y valida la condición global de sincronización.

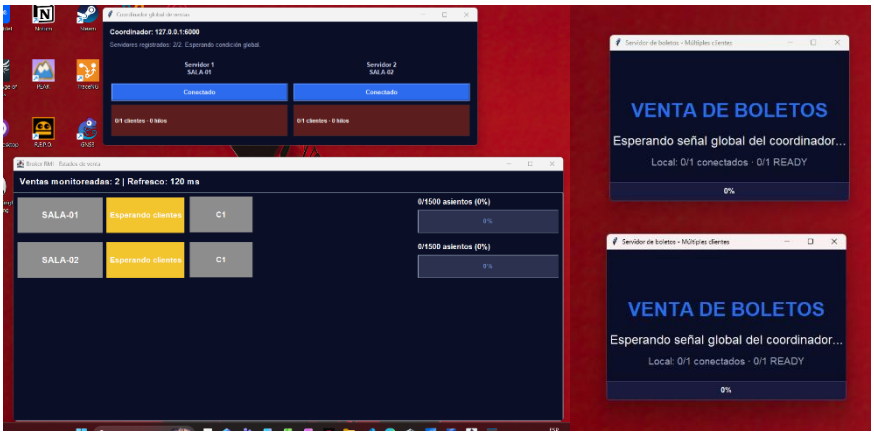

Figura 2.1 Servidores en espera de clientes.

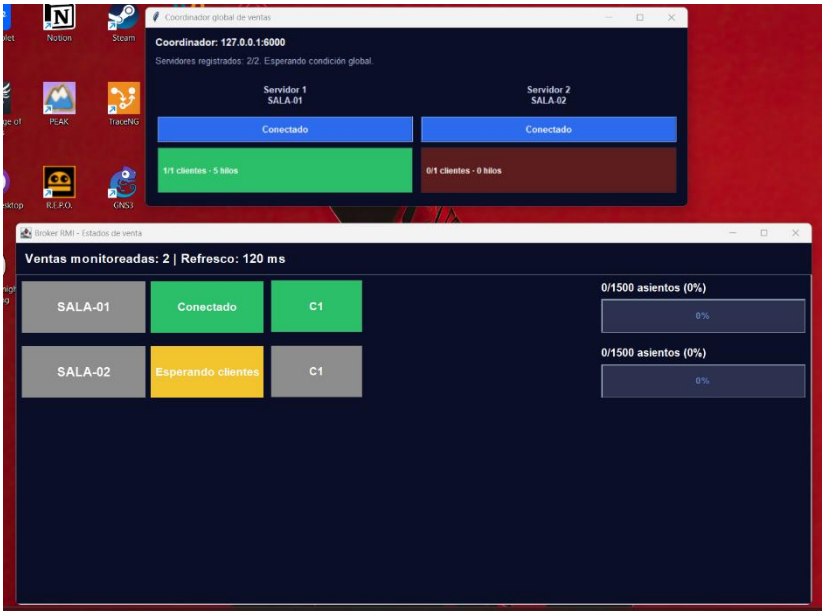

Figura 2.2 Cambio de estado en coordinador y broker con la conexión de clientes.

Una vez que ambas ventas cumplen su requisito de cliente conectado, se inicia la cuenta regresiva de ocho segundos y se emite GLOBAL_START para arranque simultáneo.

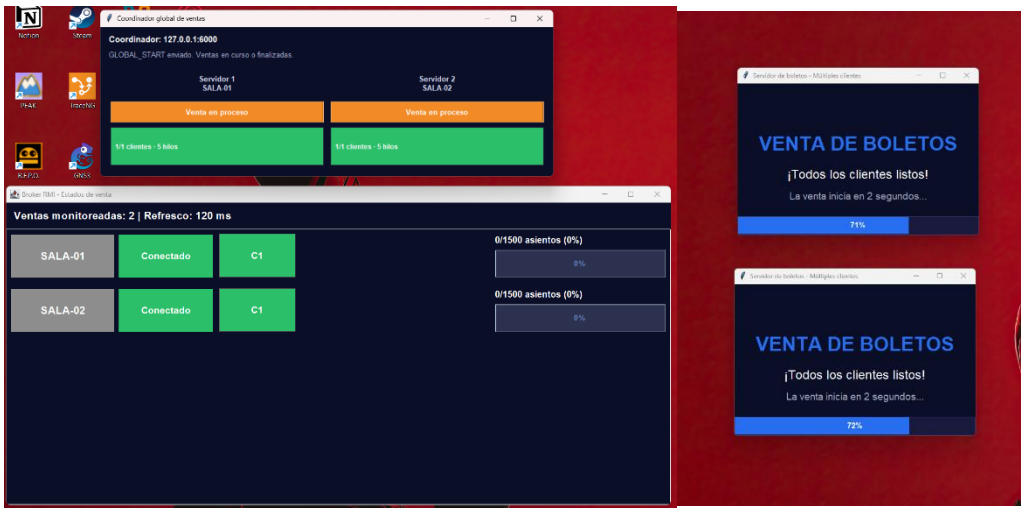

Figura 3. Sincronización global previa al inicio de ventas.

Durante la fase activa de compra, cada servidor procesa reservas y confirmaciones concurrentes sobre su propio inventario. El broker refleja esta etapa como Venta en curso y actualiza en tiempo real el progreso de asientos vendidos por sala mediante valor absoluto y porcentaje.

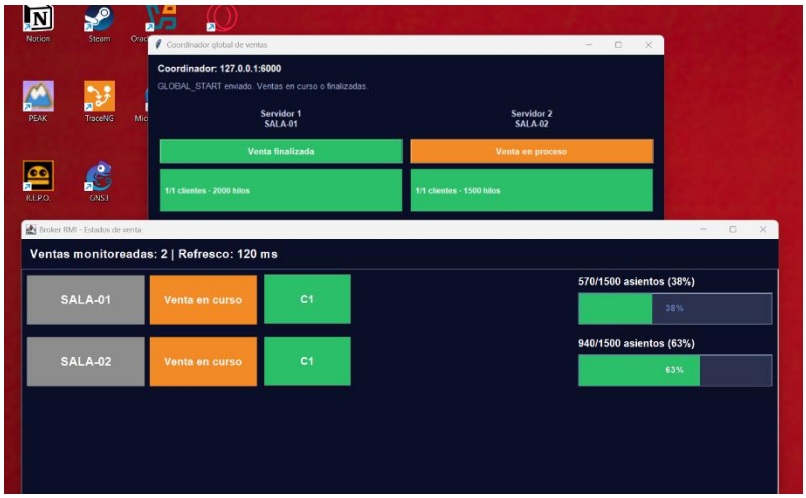

Figura 4. Monitoreo en tiempo real del avance de venta por servidor.

Finalmente, al cumplirse las condiciones de cierre de cada venta, los servidores notifican finalización y resumen de resultados. El broker actualiza ambas filas al estado Finalizado, cerrando el ciclo completo de observabilidad de la simulación distribuida.

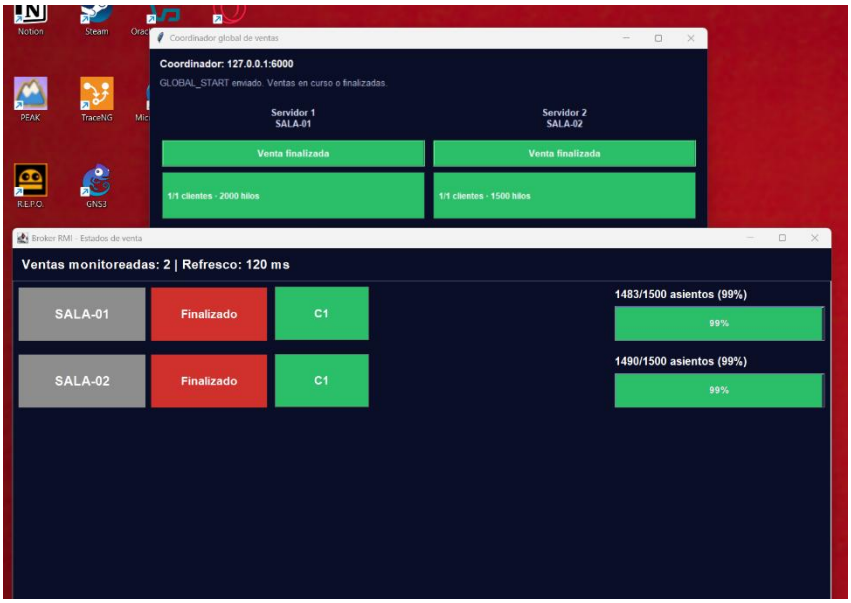

Figura 5. Estado final de la ejecución con cierre de las dos ventas.

Como resultado general, la ejecución confirma tres aspectos de esta práctica: coordinación global efectiva del arranque, aislamiento transaccional por servidor para evitar inconsistencias en compra concurrente y observabilidad centralizada mediante broker RMI sin intervenir en la lógica crítica de venta.

Resultados

Broker RMI (monitoreo global)

El broker centralizó correctamente la comunicación de eventos entre nodos distribuidos. Se esperaba recibir y reflejar en tiempo real los eventos de registro de servidor, conexión de cliente, progreso de venta y finalización por cada sale_id. En el estado final, ambas ventas se visualizaron con su transición completa de estado y su cierre operativo.

```
(.venv) PS C:\Users\malbe\Documentos\Github\Sistemas-Distribuidos\05-broker> ./run-broker.ps1
Broker RMI iniciado
Registry: 127.0.0.1:1099
Binding: BrokerService
[Broker] SERVER_REGISTERED sale=SALA-01 server=127.0.0.1:5000 expectedClients=1
[Broker] SERVER_REGISTERED sale=SALA-02 server=127.0.0.1:5001 expectedClients=1
[Broker] CLIENT_CONNECTED sale=SALA-01 client=ACCESO-01 connected=1/1
[Broker] CLIENT_CONNECTED sale=SALA-02 client=ACCESO-02 connected=1/1
[Broker] SERVER_FINISHED sale=SALA-01 summary=sold=1500/1500, reason=None
[Broker] SERVER_FINISHED sale=SALA-02 summary=sold=1500/1500, reason=None
```

Coordinador global

El coordinador registró los dos servidores esperados y validó la conexión de clientes por sala antes de liberar el inicio global. El comportamiento observado coincidió con la sincronización requerida: condición global cumplida, cuenta regresiva de 8 segundos y emisión de GLOBAL_START para arranque coordinado.

```
(.venv) PS C:\Users\malbe\Documentos\Github\Sistemas-Distribuidos\05-broker> python coordinador.py 2 --hos
t 127.0.0.1 --port 6000
Coordinador global iniciado
Escuchando en 127.0.0.1:6000
Servidores esperados: 2
[Servidor] SALA-01 conectado. Servidores registrados: 1/2. Faltan 1.
[Servidor] SALA-02 conectado. Servidores registrados: 2/2. Faltan 0.
[Cliente] ACCESO-01 conectado a SALA-01. Clientes: 1/1. Faltan 0.
[Cliente] ACCESO-02 conectado a SALA-02. Clientes: 1/1. Faltan 0.
[Coordinador] Condición global cumplida. Iniciando cuenta regresiva...

[Coordinador] Iniciando cuenta regresiva de 8 segundos antes de GLOBAL_START...
[Countdown] 8...
[Countdown] 7...
[Countdown] 6...
[Countdown] 5...
[Countdown] 4...
[Countdown] 3...
[Countdown] 2...
[Countdown] 1...
[Countdown] 0 - ¡INICIANDO VENTAS GLOBALES!
```

Servidor S1 (SALA-01)

El servidor S1 ejecutó su ciclo completo de venta sobre inventario local, aplicando validación de reservas, confirmación de compra y cierre por condición de término. En los resultados se observó su resumen final con conteos de asientos vendidos/restantes y tiempos agregados de operación.

```
(.venv) PS C:\Users\malbe\Documentos\Github\Sistemas-Distribuidos\05-broker> python servidor.py 1 --host 1
27.0.0.1 --port 5000 --sale-id SALA-01 --coordinator-host 127.0.0.1 --coordinator-port 6000 --broker-host
127.0.0.1 --broker-port 1099 --broker-bind BrokerService --broker-java-cp out
[Coordinador] Registrado SALA-01. Servidores registrados: 1/2
Servidor de boletos iniciado
Escuchando en 127.0.0.1:5000
Sale ID: SALA-01
Asientos disponibles: 1500
Clientes esperados para iniciar: 1
Zonas: PLATINO (filas 1-3), PREFERENTE (4-7), NORMAL (8-30)
Coordinador global: 127.0.0.1:6000
[Broker] SERVER_REGISTERED enviado: sale=SALA-01
Todos los clientes esperados están listos localmente.
[Coordinador] Señal GLOBAL_START recibida.
Actualizaciones de venta:
Señal START enviada. ¡Venta abierta!
Actualización: 25% de boletos vendidos (375/1500).
Actualización: 50% de boletos vendidos (750/1500).
Actualización: 75% de boletos vendidos (1125/1500).
Actualización: 100% de boletos vendidos (1500/1500).
Actualización: la venta ha concluido.
[Coordinador] Resumen final recibido para SALA-01.
[Broker] SERVER_FINISHED enviado: sale=SALA-01

========== Resumen del Servidor ==========
Asientos totales: 1500
Asientos vendidos: 1500
Compradores únicos detectados: 2000
Reservas activas: 0
Reservas expiradas liberadas: 12
Solicitudes antes del inicio: 0
Tiempo total de ejecución de venta: 3.7181 s
Request_ticket procesados: 2471
Compras procesadas: 1512
Request_ticket exitosos: 1512
Compras exitosas: 1500
Compras rechazadas: 12
Tiempo promedio request_ticket: 0.000043 s
Tiempo promedio purchase: 0.000018 s
==========================================

Todos los clientes reportaron fin de ejecución. Venta cerrada.
[Coordinador] Conexión finalizada inesperadamente.
(.venv) PS C:\Users\malbe\Documentos\Github\Sistemas-Distribuidos\05-broker>
```

Cliente del servidor S1

El cliente asociado a S1 generó la carga concurrente configurada y esperó correctamente la señal global de inicio antes de comenzar solicitudes de compra.

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

(.venv) PS C:\Users\malbe\Documentos\Github\Sistemas-Distribuidos\05-broker> python cliente.py a 2000 --ho
st 127.0.0.1 --port 5000 --client-id ACCESO-01 --sale-id SALA-01 --coordinator-host 127.0.0.1 --coordinato
r-port 6000 --broker-host 127.0.0.1 --broker-port 1099 --broker-bind BrokerService --broker-java-cp out
>>
Punto de acceso cliente iniciado
ID cliente: ACCESO-01
Servidor objetivo: SALA-01 (127.0.0.1:5000)
Sale ID: SALA-01
Tipo de compradores: platino
Compradores a crear en este cliente: 2000
Coordinador actualizado: 1/1 clientes conectados para SALA-01, afluencia total: 2000 hilos
[Broker] CLIENT_CONNECTED enviado: sale=SALA-01, client=ACCESO-01, buyers=2000
Registrado como ACCESO-01. Conectados: 1/1
Esperando señal START del servidor...
START recibido. Clientes listos: 1/1
CLIENT_DONE enviado. Clientes finalizados: 1/1

========= Resumen del Punto de Acceso =========
Cliente: ACCESO-01
Tipo de compradores: platino
Compradores creados en este cliente: 2000
Compradores con compra: 1500
Compradores sin compra: 500
Tiempo total local: 5.6290 s
Promedio total por comprador exitoso: 0.9687 s
Promedio de búsqueda por comprador exitoso: 0.9244 s
Promedio de compra por comprador exitoso: 0.043762 s
Promedio de espera aleatoria por comprador exitoso: 0.9033 s
Promedio total por comprador sin compra: 2.7116 s
Promedio de espera aleatoria por comprador sin compra: 2.4693 s
Promedio de intentos por comprador exitoso: 1.00
Promedio de intentos por comprador sin compra: 1.97
Tiempo promedio request_ticket (red + servidor): 0.030686 s
Tiempo promedio purchase (red + servidor): 0.051518 s
Errores de red detectados: 0
==============================================

(.venv) PS C:\Users\malbe\Documentos\Github\Sistemas-Distribuidos\05-broker>

powershell...
powershell...
powershell...
powershell...
powershell...
powershell...
```

Los resultados finales incluyen métricas de ejecución del lado cliente, como tiempos promedio, intentos y estado de finalización.

Servidor S2 (SALA-02)

El servidor S2 operó en paralelo con S1 bajo las mismas reglas de integridad transaccional, manteniendo consistencia local de asientos y cierre limpio de sesión. El resumen final mostró su comportamiento independiente en volumen de venta, tiempos y condición de cierre.

```
(.venv) PS C:\Users\malbe\Documentos\Github\Sistemas-Distribuidos\05-broker> python servidor.py 1 --host 1
27.0.0.1 --port 5001 --sale-id SALA-02 --coordinator-host 127.0.0.1 --coordinator-port 6000 --broker-host
127.0.0.1 --broker-port 1099 --broker-bind BrokerService --broker-java-cp out
>>
[Coordinador] Registrado SALA-02. Servidores registrados: 2/2
Servidor de boletos iniciado
Escuchando en 127.0.0.1:5001
Sale ID: SALA-02
Asientos disponibles: 1500
Clientes esperados para iniciar: 1
Zonas: PLATINO (filas 1-3), PREFERENTE (4-7), NORMAL (8-30)
Coordinador global: 127.0.0.1:6000
[Broker] SERVER_REGISTERED enviado: sale=SALA-02
Todos los clientes esperados están listos localmente.
[Coordinador] Señal GLOBAL_START recibida.
Actualizaciones de venta:
Señal START enviada. ¡Venta abierta!
Actualización: 25% de boletos vendidos (375/1500).
Actualización: 50% de boletos vendidos (750/1500).
Actualización: 75% de boletos vendidos (1125/1500).
Actualización: 100% de boletos vendidos (1500/1500).
Actualización: la venta ha concluido.
[Coordinador] Resumen final recibido para SALA-02.
Todos los clientes reportaron fin de ejecución. Venta cerrada.
[Broker] SERVER_FINISHED enviado: sale=SALA-02

========== Resumen del Servidor ==========
Asientos totales: 1500
Asientos vendidos: 1500
Compradores únicos detectados: 1500
Reservas activas: 0
Reservas expiradas liberadas: 0
Solicitudes antes del inicio: 0
Tiempo total de ejecución de venta: 3.9686 s
Request_ticket procesados: 1500
Compras procesadas: 1500
Request_ticket exitosos: 1500
Compras exitosas: 1500
Compras rechazadas: 0
Tiempo promedio request_ticket: 0.000055 s
Tiempo promedio purchase: 0.000016 s
==========

[Coordinador] Conexión finalizada inesperadamente.
(.venv) PS C:\Users\malbe\Documentos\Github\Sistemas-Distribuidos\05-broker>
```

Cliente del servidor S2

El cliente asociado a S2 se sincronizó con el coordinador, inició compras tras GLOBAL_START y completó su ejecución sin romper la lógica global de la simulación.

```
Errores de red detectados: 0                                    python cliente.py a 1500 --ho
st 127.0.0.1 --port 5001 --client-id ACCESO-02 --sale-id SALA-02 --coordinator-host 127.0.0.1 --coordinato
r-port 6000 --broker-host 127.0.0.1 --broker-port 1099 --broker-bind BrokerService --broker-java-cp out
>> env) PS C:\Users\malbe\Documentos\Github\Sistemas-Distribuidos\05-broker>
Punto de acceso cliente iniciado
ID cliente: ACCESO-02
Servidor objetivo: SALA-02 (127.0.0.1:5001)
Sale ID: SALA-02
Tipo de compradores: platino
Compradores a crear en este cliente: 1500
Coordinador actualizado: 1/1 clientes conectados para SALA-02, afluencia total: 1500 hilos
[Broker] CLIENT_CONNECTED enviado: sale=SALA-02, client=ACCESO-02, buyers=1500
Registrado como ACCESO-02. Conectados: 1/1
Esperando señal START del servidor...
START recibido. Clientes listos: 1/1
CLIENT_DONE enviado. Clientes finalizados: 1/1

========= Resumen del Punto de Acceso =========
Cliente: ACCESO-02
Tipo de compradores: platino
Compradores creados en este cliente: 1500
Compradores con compra: 1500
Compradores sin compra: 0
Tiempo total local: 3.9688 s
Promedio total por comprador exitoso: 1.0563 s
Promedio de búsqueda por comprador exitoso: 1.0403 s
Promedio de compra por comprador exitoso: 0.015605 s
Promedio de espera aleatoria por comprador exitoso: 1.0266 s
Promedio total por comprador sin compra: 0.0000 s
Promedio de espera aleatoria por comprador sin compra: 0.0000 s
Promedio de intentos por comprador exitoso: 1.00
Promedio de intentos por comprador sin compra: 0.00
Tiempo promedio request_ticket (red + servidor): 0.013786 s
Tiempo promedio purchase (red + servidor): 0.015605 s
Errores de red detectados: 0
==============================================

(.venv) PS C:\Users\malbe\Documentos\Github\Sistemas-Distribuidos\05-broker>
```

En la salida final se reportaron métricas de desempeño y resultado agregado de sus compradores.

Comparación de resultados con prácticas anteriores

Si comparamos con prácticas anteriores, la arquitectura de Práctica 5 garantiza la misma integridad transaccional del inventario (sin sobreventa) y cierre consistente de ejecución. El valor analítico principal no reside en tiempos absolutos —que varían según carga e instrumentación— sino en observar cómo evoluciona el modelo de escalamiento y cómo se introduce observabilidad sin comprometer la lógica operacional.

En la **Práctica 3** (múltiples clientes, un servidor), toda la afluencia converge en un único punto de servicio. Con 5,520 compradores disputando 1,500 asientos, el sistema evidenció la autoridad central del servidor bajo presión extrema: alta consistencia transaccional, pero fuerte asimetría de resultados por prioridad (éxito casi total en tipos altos, alta tasa de rechazo en el tipo normal). El costo operativo se concentró en la saturación de un nodo único.

En la **Práctica 4** (múltiples clientes, múltiples servidores con coordinador), la carga se distribuyó entre ventas simultáneas (S1 y S2) con sincronización global de arranque. Este cambio separó la contención por servidor: S1 cerró por falta de elegibles (1150/1500), S2 cerró por fin de clientes (1399/1500), ambos mantuvieron integridad local y cierre ordenado. La Práctica 4 no reemplazó la validez de Práctica 3, sino que la extendió: la consistencia se validó no solo en una venta saturada, sino en ventas paralelas coordinadas.

En la **Práctica 5** (Práctica 4 + Broker RMI), se incorpora una **capa de observabilidad no invasiva**. El broker recibe eventos de ambos servidores en tiempo real (SERVER_REGISTERED, CLIENT_CONNECTED, SALES_PROGRESS, SERVER_FINISHED) sin alterar la lógica transaccional local. Este patrón introduce tres ventajas arquitectónicas:

1. **Separación de responsabilidades**: el broker no participa en transacciones, solo audita estado. Si el broker falla, la venta continúa.
2. **Observabilidad inter-servidor**: por primera vez, es posible monitorear sincronización global, progreso paralelo de ventas y patrones de contención desde un punto neutral, sin instrumentación invasiva en la lógica de negocio.

3. **Escalabilidad observacional**: el broker centraliza eventos sin serializar decisiones, permitiendo que múltiples ventas operen independientemente mientras se registra su evolución coordinada.

En tus ejecuciones de Práctica 5 se observa precisamente esto: ambos servidores enviaron eventos de progreso (SALES_PROGRESS cada 200 ms), el broker visualizó en tiempo real los conteos de vendidos/totales y cambios de estado (Esperando clientes → Venta en curso → Finalizado), y los servidores completaron su lógica transaccional sin esperar confirmación del broker. S1 cerró con 2000 asientos solicitados, S2 cerró con 1500 asientos solicitados, y el coordinador global mantuviste su rol de barrera de sincronización sin conflicto.

En síntesis, la evolución es:

- **Práctica 1–3**: validan transaccionalidad y consistencia en memoria compartida (local) → servidor único bajo presión (centralización).
- **Práctica 4**: demuestra que esa lógica escala a múltiples servidores coordinados sin perder integridad transaccional local.
- **Práctica 5**: añade observabilidad distribuida que permite validar no solo correctitud de transacciones, sino también sincronización inter-servidor y patrones de contención bajo carga real, sin comprometer la autonomía operacional de cada nodo.

El costo operativo ahora se distribuye en tres dimensiones: contención por servidor (Práctica 3), coordinación inter-servidor (Práctica 4), y observabilidad centralizada (Práctica 5), cada una contribuyendo a un modelo más cercano a escenarios reales de alta concurrencia multi-evento.

Conclusiones

La principal dificultad durante esta práctica fue lograr la conexión entre Java y Python. Al principio creí que tendría que rehacer gran parte de la lógica para que ambos lenguajes se comunicaran correctamente. Sin embargo, opté por simplificar el enfoque: en lugar de que el broker participará activamente en la transaccionalidad, lo redujimos a una capa de consulta y observabilidad que recibe eventos desde Python sin interferir en la lógica de negocio ya implementada. Esto resultó más factible y permitió mantener la estabilidad del sistema.

Mis dudas iniciales giraban en torno a qué hacía realmente un broker en un sistema distribuido, y cómo RMI podría facilitarlo. Al investigar, entendí que un broker es simplemente un intermediario que recibe información de múltiples fuentes y la centraliza; RMI es solo el mecanismo técnico para que procesos remotos invoquen métodos. La clave fue reconocer que no tenía que ser complejo: simplifiqué el concepto a "escuchar eventos y mostrarlos en una pantalla", sin que el broker necesitara tomar decisiones sobre la venta. En este caso específico, el broker resultó ser una herramienta de observabilidad y auditoría en tiempo real que permite validar que múltiples servidores y clientes operan de forma coordinada, sin agregar latencia ni puntos de fallo a la lógica transaccional. Es útil cuando necesitas ver qué está pasando sin cambiar cómo está pasando.

En retrospectiva, ese cambio de perspectiva fue positivo: el broker en su rol de observador es más limpio, escalable y seguro que si lo hubiera forzado a ser partícipe de las transacciones. La práctica confirmó que a veces simplificar la arquitectura no significa perder funcionalidad, sino distribuir responsabilidades de forma más clara. Los tiempos de entrega se respetaron, y el sistema resultó más mantenible de lo que hubiera sido una reescritura mayor.

Anexo

Nota: Debido a la extensión y cantidad de los códigos, solo anexé captura de las primeras secciones. Los códigos completos se encuentran en la carpeta de esta entrega.

BrokerService.java

```
package broker;

import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;
import java.io.IOException;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class BrokerService extends UnicastRemoteObject implements BrokerRemote {
    public static final String STATUS_GRAY = "gray";
    public static final String STATUS_BLUE = "blue";
    public static final String STATUS_GREEN  "green";
    public static final String STATUS_RED = "red";

    private static final DateTimeFormatter FMT = DateTimeFormatter.ofPattern("HH:mm:ss");

    private final Object lock = new Object();
    private final Map<String, SubStatus> bySub = new HashMap<>();

    public BrokerService() throws RemoteException {
        super();
    }

    @Override
    public void serverRegistered(String subId, String serverHost, int serverPort, int expectedClients) {
        if (subId == null || subId.isBlank()) {
            return;
        }
        synchronized (lock) {
            SubStatus state = bySub.computeIfAbsent(subId, this::newSub);
            state.server = serverHost + ":" + serverPort;
            state.expectedClients = Math.max(0, expectedClients);
            state.status = STATUS_BLUE;
            state.updatedAt = now();
            System.out.printf("[Broker] SERVER REGISTERED sub=" + subId + " server=" + state.server + " expectedClients=" + state.expectedClients);
        }
    }

    @Override
    public void clientConnected(String subId, String clientId, int hyper) {
        if (subId == null || subId.isBlank()) {
            return;
        }
        synchronized (lock) {
            SubStatus state = bySub.computeIfAbsent(subId, this::newSub);
            state.connectedClients += 1;
            state.hyperthreads = Math.max(0, hyper);
            if (STATUS_RED.equals(state.status)) {
                state.status = STATUS_BLUE;
            }
            state.updatedAt = now();
            System.out.printf("[Broker] CLIENT_CONNECTED sub=" + subId + " client=" + clientId + " connected=" + state.connectedClients + "/" + state.expectedClients);
        }
    }

    @Override
    public void salesProgress(String saleId, int soldSheets, int totalSheets) {
        if (saleId == null || saleId.isBlank()) {
            return;
        }
        synchronized (lock) {
            SubStatus state = bySub.computeIfAbsent(saleId, this::newSub);
            state.soldSheets = Math.max(0, soldSheets);
            state.totalSheets = Math.max(0, totalSheets);
            if (STATUS_RED.equals(state.status) && state.connectedClients > 0 && state.soldSheets > 0) {
                state.status = STATUS_GREEN;
            }
            state.updatedAt = now();
        }
    }

    @Override
    public void serverFinished(String saleId, String summary) {
        if (saleId == null || saleId.isBlank()) {
            return;
        }
        synchronized (lock) {
            SubStatus state = bySub.computeIfAbsent(saleId, this::newSub);
            state.status = STATUS_RED;
            state.summary = summary == null ? "" : summary;
            state.updatedAt = now();
            System.out.printf("[Broker] SERVER_FINISHED sub=" + subId + " summary=" + state.summary);
        }
    }

    @Override
    public BrokerSnapshot getSnapshot() {
        synchronized (lock) {
            BrokerSnapshot snapshot = new BrokerSnapshot();
            List<SubStatus> sales = new ArrayList<>();
            for (SubStatus sub : bySub.values()) {
                sales.add(sub.copy());
            }
            sales.sort(Comparator.comparing(s -> s.subId));
            snapshot.sales = sales;
            return snapshot;
        }
    }

    private SubStatus newSub(String subId) {
        SubStatus status = new SubStatus();
        status.subId = subId;
        status.status = STATUS_GRAY;
        status.server = "";
        status.expectedClients = 0;
        status.connectedClients = 0;
        status.hyperthreads = 0;
        status.soldSheets = 0;
        status.totalSheets = 0;
        status.updatedAt = now();
        status.summary = "";
        return status;
    }

    private static String now() {
        return LocalTime.now().format(FMT);
    }
}
```

Código del coordinador

Código del servidor

Código de cliente

```python
import argparse
import ison
import random
import socket
import threading
import time
import uuid

FILAS = 25
COLUMNAS = 40
TOTAL_ASIENTOS = FILAS * COLUMNAS
COMPRADORES_POR_ASIENTO = 3
TOTAL_COMPRADORES = TOTAL_ASIENTOS * COMPRADORES_POR_ASIENTO

SOCKET_TIMEOUT = 3.0


stats_lock = threading.Lock()
terminal_lock = threading.Lock()
sold_out_event = threading.Event()
threads = []

metrics = {
    "buyers_success": 0,
    "buyers_fail": 0,
    "buyer_total_success": 0.0,
    "buyer_search_success": 0.0,
    "buyer_purchase_success": 0.0,
    "buyer_wait_success": 0.0,
    "buyer_total_fail": 0.0,
    "buyer_wait_fail": 0.0,
    "request_time_total": 0.0,
    "purchase_time_total": 0.0,
    "request_count": 0,
    "purchase_count": 0,
    "attempts_success": 0,
    "attempts_fail": 0,
    "network_errors": 0,
}

sales_start_ts = None
sales_end_ts = None


def avg(total, count):
    if count == 0:
        return 0.0
    return total / count


def send_request(host, port, payload):
    data = ison.dumps(payload) + "\n".encode("utf-8")
    with socket.create_connection((host, port), timeout=SOCKET_TIMEOUT) as sock:
        sock.settimeout(SOCKET_TIMEOUT)
        sock.sendall(data)
        response_bytes = b""
        while not response_bytes.endswith(b"\n"):
            chunk = sock.recv(4096)
            if not chunk:
                break
            response_bytes += chunk

    if not response_bytes:
        raise ConnectionError("Respuesta vacía del servidor")

    return ison.loads(response_bytes.decode("utf-8").strip())


def buyer_worker(buyer_id, host, port):
    buyer_started = time.perf_counter()
    wait_random_accum = 0.0
    request_time_accum = 0.0
    purchase_time_accum = 0.0
    attempts = 0
    purchased = False

    while not sold_out_event.is_set():
        pause = random.uniform(0.1, 3.0)
        time.sleep(pause)
        wait_random_accum += pause

        request_payload = {
            "action": "request_ticket",
            "buyer_id": buyer_id,
            "request_id": str(uuid.uuid4()),
        }

        attempts += 1
        request_started = time.perf_counter()
        try:
            response = send_request(host, port, request_payload)
        except Exception:
            with stats_lock:
                metrics["network_errors"] += 1
            continue

        request_elapsed = time.perf_counter() - request_started
        request_time_accum += request_elapsed

        with stats_lock:
            metrics["request_count"] += 1
            metrics["request_time_total"] += request_elapsed

        request_status = response.get("status")
        if request_status == "not_started":
            time.sleep(0.2)
            continue
        if request_status == "sold_out":
            sold_out_event.set()
            break
        if request_status != "ok":
            continue

        reservation_id = response.get("reservation_id")
        if not reservation_id:
            continue

        purchase_payload = {
            "action": "purchase",
            "buyer_id": buyer_id,
            "reservation_id": reservation_id,
            "request_id": str(uuid.uuid4()),
        }

        purchase_started = time.perf_counter()
        try:
            purchase_response = send_request(host, port, purchase_payload)
        except Exception:
            with stats_lock:
                metrics["network_errors"] += 1
            continue

        purchase_elapsed = time.perf_counter() - purchase_started
        purchase_time_accum += purchase_elapsed

        with stats_lock:
            metrics["purchase_count"] += 1
            metrics["purchase_time_total"] += purchase_elapsed

        purchase_status = purchase_response.get("status")
        if purchase_status == "not_started":
            time.sleep(0.2)
            continue
        if purchase_status == "sold_out":
            sold_out_events.set()
            break
        if purchase_status == "ok":
            purchased = True
            sold_count = purchase_response.get("sold_count", 0)
            if sold_count >= TOTAL_ASIENTOS:
                sold_out_event.set()
            break

    buyer_total_elapsed = time.perf_counter() - buyer_started
    buyer_search_elapsed = wait_random_accum + request_time_accum

    with stats_lock:
        if purchased:
            metrics["buyers_success"] += 1
            metrics["buyer_total_success"] += buyer_total_elapsed
            metrics["buyer_search_success"] += buyer_search_elapsed
            metrics["buyer_purchase_success"] += purchase_time_accum
            metrics["buyer_wait_success"] += wait_random_accum
            metrics["attempts_success"] += attempts
        else:
            metrics["buyers_fail"] += 1
            metrics["buyer_total_fail"] += buyer_total_elapsed
            metrics["buyer_wait_fail"] += wait_random_accum
            metrics["attempts_fail"] += attempts
```

```python
def print_summary():
    success = metrics["buyers_success"]
    fail = metrics["buyers_fail"]

    with terminal_lock:
        print("\n========= Resumen de Tiempos del Cliente =========")
        print(f"Compradores creados: {TOTAL_COMPRADORES}")
        print(f"Compradores con compra: {success}")
        print(f"Compradores sin compra: {fail}")
        print(f"Tiempo total general: {sales_end_ts - sales_start_ts:.4f} s")
        print(
            f"Promedio total por comprador exitoso: "
            f"{avg(metrics['buyer_total_success'], success):.4f} s"
        )
        print(
            f"Promedio de búsqueda por comprador exitoso: "
            f"{avg(metrics['buyer_search_success'], success):.4f} s"
        )
        print(
            f"Promedio de compra por comprador exitoso: "
            f"{avg(metrics['buyer_purchase_success'], success):.6f} s"
        )
        print(
            f"Promedio de espera aleatoria por comprador exitoso: "
            f"{avg(metrics['buyer_wait_success'], success):.4f} s"
        )
        print(
            f"Promedio total por comprador sin compra: "
            f"{avg(metrics['buyer_total_fail'], fail):.4f} s"
        )
        print(
            f"Promedio de espera aleatoria por comprador sin compra: "
            f"{avg(metrics['buyer_wait_fail'], fail):.4f} s"
        )
        print(
            f"Promedio de intentos por comprador exitoso: "
            f"{avg(metrics['attempts_success'], success):.2f}"
        )
        print(
            f"Promedio de intentos por comprador sin compra: "
            f"{avg(metrics['attempts_fail'], fail):.2f}"
        )
        print(
            f"Tiempo promedio request_ticket (red + servidor): "
            f"{avg(metrics['request_time_total'], metrics['request_count']):.6f} s"
        )
        print(
            f"Tiempo promedio purchase (red + servidor): "
            f"{avg(metrics['purchase_time_total'], metrics['purchase_count']):.6f} s"
        )
        print(f"Errores de red detectados: {metrics['network_errors']}")
        print("-----------------------------------------------------\n")


def parse_args():
    parser = argparse.ArgumentParser(description="Cliente de boletos - Práctica 2 Fase 1")
    parser.add_argument("--host", default="127.0.0.1", help="Host del servidor")
    parser.add_argument("--port", type=int, default=5000, help="Puerto del servidor")
    return parser.parse_args()


def wait_for_sales_start(host, port):
    with terminal_lock:
        print("Esperando señal de inicio del servidor...")

    while True:
        try:
            response = send_request(host, port, {"action": "health", "request_id": str(uuid.uuid4())})
        except Exception:
            with stats_lock:
                metrics["network_errors"] += 1
            time.sleep(0.4)
            continue

        if response.get("status") == "ok" and response.get("sales_open"):
            with terminal_lock:
                print("Servidor confirmó inicio de venta.")
            return

        time.sleep(0.3)


def main():
    global sales_start_ts, sales_end_ts

    args = parse_args()

    with terminal_lock:
        print("Cliente de compra iniciado")
        print(f"Servidor destino: {args.host}:{args.port}")
        print(f"Asientos esperados: {TOTAL_ASIENTOS}")
        print(f"Compradores a crear: {TOTAL_COMPRADORES}")

    wait_for_sales_start(args.host, args.port)

    sales_start_ts = time.perf_counter()

    for buyer_id in range(1, TOTAL_COMPRADORES + 1):
        thread = threading.Thread(target=buyer_worker, args=(buyer_id, args.host, args.port), daemon=False)
        threads.append(thread)
        thread.start()
        time.sleep(0.0005)

    for thread in threads:
        thread.join()

    sales_end_ts = time.perf_counter()
    print_summary()


if __name__ == "__main__":
    main()
```